

1 INTRODUCTION

Gradually typed languages provide a unified setting in which dynamic and statically typed programming styles can be accommodated along with free interoperability between static and dynamically typed components.

The metatheory of gradually typed languages is considered especially challenging, as it inherently involves (1) general recursion and at least an error effect distinct from non-termination (2) the verification of type-based equational reasoning to demonstrate the soundness of static typing and (3) proving graduality, a relational property of program semantics that establishes that increasing the amount of static typing in a program results in a refinement of program behavior. Most prior work to this metatheory relies on purely operational approaches: an operational semantics for the language is developed and a simulation-based argument is given for graduality. These have the advantage of being fairly elementary and so amenable to direct mechanization in a proof assistant but at the cost of lacking reusability. Furthermore, equational aspects of the language are for the most part ignored. A second line of prior work is to develop generic operational frameworks in which the above can be carried out: the gradualizer as well as Abstracting Gradual Typing. These are reusable techniques that apply across languages but still do not address equational theory and . Finally a third line of prior work gives a semantic theory of gradual typing based on the idea that casts form embedding-projection pairs in a 2-categorical semantics. This approach has been formalized in two ways: one using classical denotational semantics based on complete partial orders (CPOs) and another based on step-indexed logical relations over an untyped operational semantics. This approach has similar benefits of abstraction to the AGT framework, but suffers from a lack of reusability. First, classical domain theory is believed to be unable to model higher-order references, a feature of all practical gradually typed languages. Second, the proofs based on step-indexed logical relations, while known to scale to higher-order references are relatively unstructured and complex. While inspired by the embedding-projection pair semantics, several components of the proof that work in the CPO-based semantics break down in the step-indexed setting for unclear reasons. This work also tends to have a high level of tedious proof overhead in practice as evidenced that most papers using this approach include tech reports of repetitive paper proofs that typically dwarf the size of the paper itself.

In this current work it is our goal to combine the benefits of the previous approaches: we desire the compositionality and reusability of the domain theoretic semantics, the genericity of abstracting gradual typing and the gradualizer, the easy mechanizability of the elementary operational techniques as well as the generality of step-indexed approaches.

2 CALL-BY-VALUE GRADUAL CAST CALCULUS: SYNTAX AND AXIOMS

2.1

3 A COMPOSITIONAL “OPERATIONAL” SEMANTICS

We start by giving a concrete description of the semantics our denotational approach provides in the style of a monadic interpreter.

The values of a syntactic type A in the GCC are interpreted as elements of a set $\llbracket A \rrbracket$: values of number type are interpreted as numbers, values of function type are interpreted as functions with a monadic output and value of the dynamic type are interpreted in a complex set that contains numbers and functions as subsets.

For now for concreteness we will work with a simple monad that supports only errors and divergence with observable computational steps. This can be described as the composition of two monads $TX = L(X_{\perp})$ where LX is Capretta’s coinductive delay monad and $X_{\perp}$ is simply the maybe monad where we call the additional element $\perp$ to connote that it is intended to represent a runtime

error. Capretta's delay monad is a coinductive set defined as the greatest solution to the equation $LX \cong \text{Done}(X) + \text{Running}(LX)$. That is an element is either done with a produced X or takes an observable computational step. Taking the greatest fixed point rather than least fixed point allows for the possibility of a program that runs forever. We think of this as a computational process that requires external "fuel" to execute: either the process is complete and we receive a final value or the computation requires an additional fuel to continue executing.

For each computation $\Gamma \vdash M : A$ we define a 4-place relation $\gamma \vDash M \Downarrow^n V$ where $\gamma \in \llbracket \Gamma \rrbracket$, $n \in \mathbb{N}$ and $V \in \llbracket A \rrbracket$. Further this relation is a decidable partial function when considering M, γ, n as inputs and V as an output and is a partial function when considering M, γ as inputs and n as an output.

$$\gamma \vDash \text{zro} \Downarrow^0 0 \qquad \frac{\gamma \vDash M \Downarrow^n m}{\gamma \vDash \text{suc}(M) \Downarrow^n m+1} \qquad \frac{R(V_i, n, V_o) \iff (\gamma, x \mapsto V_i) \vDash M \Downarrow^n V_o}{\gamma \vDash \lambda x. M \Downarrow^0 R}$$

As well as two

of values, computations, upcasts and downcasts in GCC mutually recursively

3.1 Abstracting GCC Semantics

4 DISCUSSION

4.1 Synthetic Ordering

While the use of synthetic guarded domain theory allows us to very conveniently work with non-well-founded recursive constructions while abstracting away the precise details of step-indexing, we do work with the error ordering in a mostly analytic fashion in that gradual types are interpreted as sets equipped with an ordering relation, and all terms must be proven to be monotone. It is possible that a combination of synthetic guarded domain theory with *directed* type theory would allow for an a synthetic treatment of the error ordering as well.